# HW 1: Language Models from Scratch (Part 1)

Giovanni Cascone, Student ID 020046702, San José State University

Code and results: github.com/giovannicascone-4/DL_llm-hw1

**Setup.** All runs use the course code nanolm without changes (commit a4d756e) on Tiny Shakespeare with the character tokenizer. Every model gets the same budget: 2,000 steps × 8,192 tokens = 16.4M tokens, with seeds 1337 and 42. Hardware: MacBook Pro M5 Max (MPS, fp32). All 32 runs took 19.5 minutes in total.

## 1. Metrics by hand

Trained character model: modern Transformer, seed 1337. From metrics.json: val_loss = 1.467075, val_tokens = 111,360, val_bytes = 111,360.

| metric | formula | my value | logged |
|---|---|---|---|
| loss (nats/token) | val_loss | 1.467075 | 1.467075 |
| perplexity | e^loss | 4.3365 | 4.3365 (val_ppl) |
| bits/token | loss / ln 2 | 2.1165 | — |
| bits-per-byte | bits/token × val_tokens / val_bytes | 2.1165 | 2.1165 (val_bpb) |

Everything matches the logged values exactly. Since one character is one byte here, bits/token and bpb are the same number.

Untrained models (`train.max_steps=1`), average of the two seeds:

| tokenizer | V | ln V | untrained loss | loss / ln V |
|---|---|---|---|---|
| char | 65 | 4.174 | 4.168 | 1.00 |
| byte | 256 | 5.545 | 5.442 | 0.98 |
| BPE | 512 | 6.238 | 6.227 | 1.00 |
| word | 2048 | 7.625 | 7.568 | 0.99 |

An untrained model should be close to ln V, because it gives every token about the same probability, 1/V. All four are within 2% of ln V. They are slightly below it because `max_steps=1` already makes one update before evaluation.

## 2. RNN vs. LSTM vs. GRU vs. Transformer

| model | parameters | val bpb (mean ± std, 2 seeds) |
|---|---|---|
| Transformer, modern (RMSNorm, RoPE, SwiGLU) | 3.16M | 2.116 ± 0.001 |
| LSTM 2 × 256 | 1.07M | 2.137 ± 0.005 |
| GRU 2 × 256 | 0.81M | 2.160 ± 0.002 |
| LSTM 2 × 512 | 4.24M | 2.172 ± 0.009 |
| Transformer, GPT-2 style | 3.44M | 2.185 ± 0.002 |
| vanilla RNN 2 × 256 | 0.28M | 2.437 ± 0.007 |

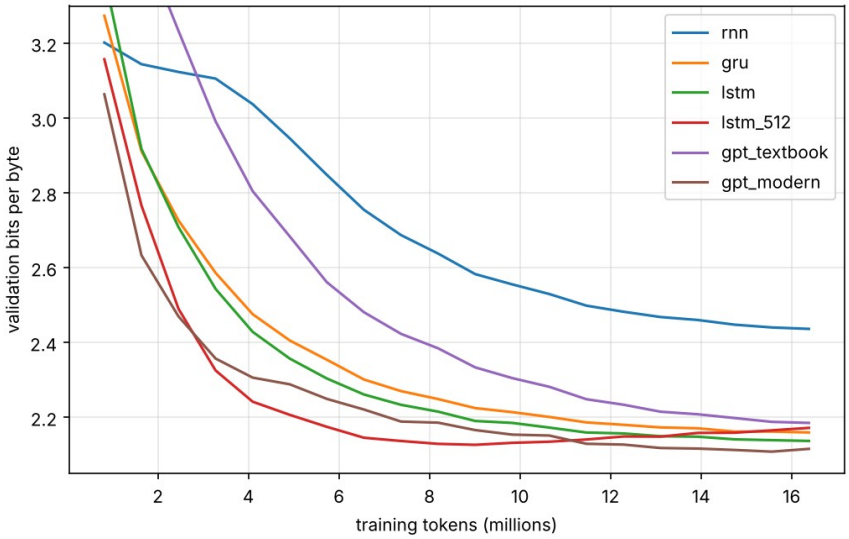



Figure 1. Validation bpb vs. training tokens (mean of 2 seeds).

## 3. Context length

| seq_len | LSTM bpb | Transformer bpb |
|---|---|---|
| 64 | 2.181 ± 0.005 | 2.156 ± 0.001 |
| 128 | 2.149 ± 0.003 | 2.114 ± 0.009 |
| 256 | 2.137 ± 0.005 | 2.116 ± 0.001 |
| 512 | 2.130 ± 0.006 | 2.103 ± 0.001 |

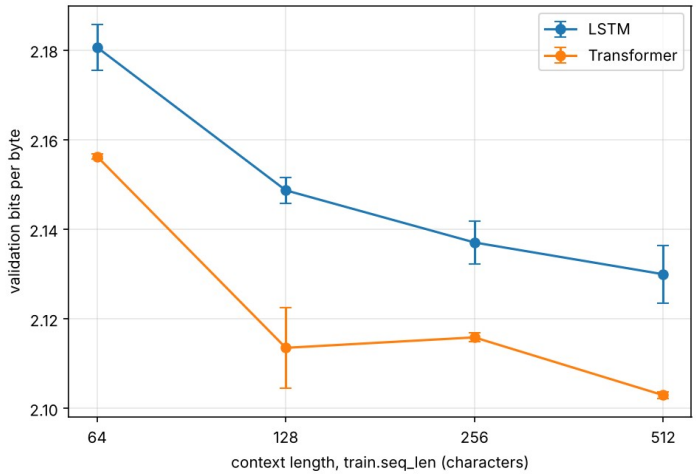



Figure 2. Validation bpb vs. context length (same token budget for every run, mean ± std of 2 seeds).

**LSTM:** stops benefiting at about 256. Going from 256 to 512 gains only 0.007, which is within the seed noise (± 0.005).

**Transformer:** still improving at 512, which gains 0.013 over 256. The flat step from 128 to 256 is noise, since the two 128 runs differ by ± 0.009.

## 4. Attention analysis

I used nanolm's attention visualization (`nanolm.viz attention`) on the trained modern Transformer. `part4_attention.py` runs it on each text and computes the averages. The three texts are Shakespeare dialogue ("ROMEO: But soft! what light through yonder window breaks? JULIET: Ay me!"), modern English

("The engineer trained a small language model on her laptop, and it wrote poems.") and a random string repeated three times. For each head I averaged the attention weight on the relevant position over all positions in the text, as in §1.6.2. For comparison, spreading attention evenly over the earlier positions would give only 0.05–0.07.

| head | type | relevant position | Shakespeare | modern | repeated |
|---|---|---|---|---|---|
| L2·H2 | previous-token | t − 1 | 0.49 | 0.57 | 0.64 |
| L2·H0 | word-start | first letter of the word | 0.27 | 0.26 | 0.34 |
| L3·H2 | line-start | last ':' or newline | 0.37 | — | — |

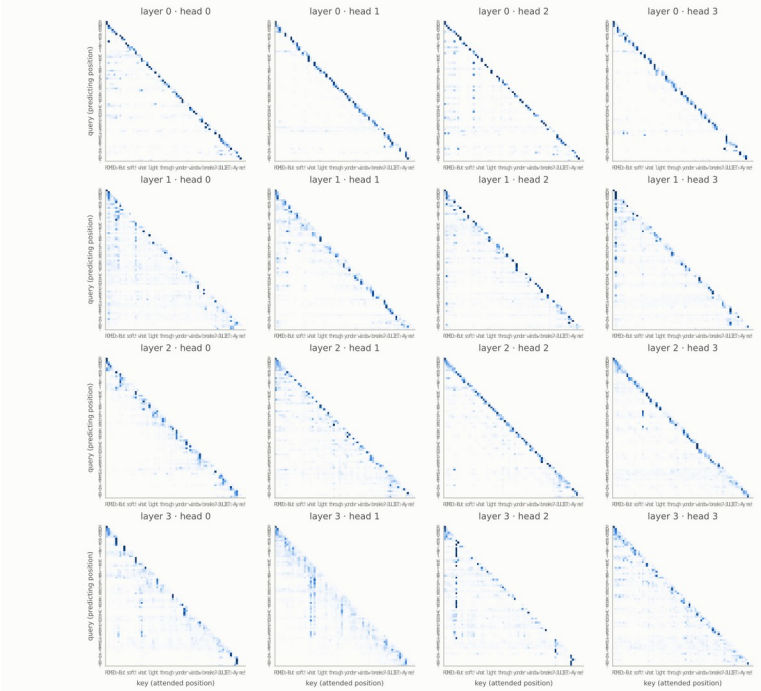



Figure 3. Attention maps of all 16 heads on the Shakespeare text. L2·H2 is a clean diagonal just below the main one (previous character). L3·H2 has vertical stripes at the ':' and newline after ROMEO.

The previous-token and word-start heads show up on all three texts with similar weights, so they are real patterns and not specific to one sentence. The line-start head only applies to the Shakespeare text, because the other two have no ':' or newline.

## Questions

**Why does the vanilla RNN fall so far behind?** It has no gates. The gradient going back through time is multiplied by the same weight matrix at every step, so it vanishes after a few characters and the model only learns short patterns. It is not overfitting: its train and validation loss are close (gap 0.06, the smallest of all models). It simply can't fit the data well. GRU and LSTM add gates and an additive memory path that lets the gradient survive, and they are about 0.3 bpb better with only 3–4× more parameters.

**Is the larger LSTM better or worse, and what in your curves shows why?** Worse. It ends at 2.172 bpb vs. 2.137 for the small LSTM. In Figure 1, its curve reaches the best value of any recurrent model (about 2.12 at 9M tokens) and then goes back up. Figure 4 shows the reason: after about 9M tokens its validation loss rises while its training loss keeps going down, to 1.03. The gap between them is 0.48, against 0.19 for the small LSTM. With 4.2M parameters and only 1 MB of text, it starts memorizing the training set (overfitting). Stopping early at about 9M tokens would have made it the best LSTM.

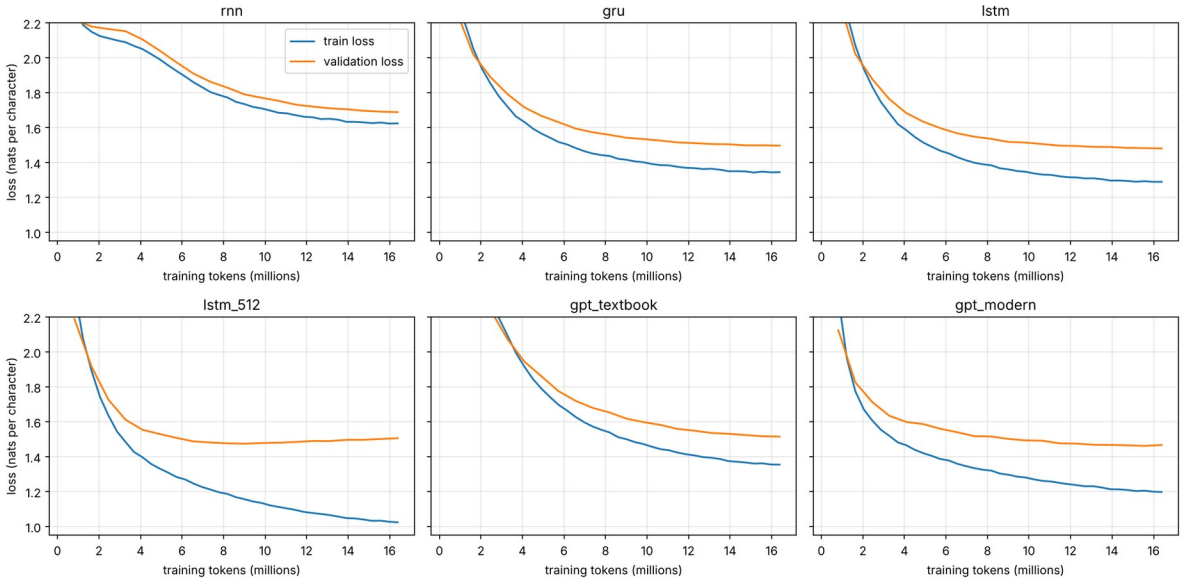



Figure 4. Training vs. validation loss (nats/char) for each model. Only the large LSTM's validation loss turns back up.

**Which architecture uses longer context more effectively?** The Transformer. It is better than the LSTM at every context length and keeps improving up to 512, while the LSTM levels off after 256. Attention can look directly at any earlier character, while the LSTM has to squeeze everything into a fixed-size state.

## Extra credit: GRU cell gate by gate

I wrote the GRU cell gate by gate, like LSTMScratch (`extra_credit_gru.py`):

$r = \sigma(W_{ir} x + b_{ir} + W_{hr} h + b_{hr})$, $z = \sigma(W_{iz} x + b_{iz} + W_{hz} h + b_{hz})$,
$n = \tanh(W_{in} x + b_{in} + r \odot (W_{hn} h + b_{hn}))$, $h' = (1 - z) \odot n + z \odot h$

I copied the weights from `nn.GRU` and compared the outputs and gradients over 50 time steps. The largest difference was 2e-16 for the outputs and 6e-14 for the gradients in float64, and about 1e-7 for the outputs in float32. That is just rounding error, so the two match.